

Project Overview

We will build **NIRVAN '26** as a fully frontend React site (no backend) with static data (e.g. `events.js`, `speakers.js`, etc.) and deploy it to Netlify. The goal is a **simple, responsive multi-page site** that presents the fest clearly and drives event discovery and registration, while using the “magical academy” aesthetic **only as visual flavor**.

- **Key Objectives:** Communicate “What is NIRVAN? Why join? What events exist? How to register?” within seconds [27†L4-L12] [15†L9-L16] .
- **Scope:** Hero, About, Events (listing + detail), Schedule, Speakers, Gallery, Contact, and a Registration call-to-action (page or modal). All data is **hard-coded/static** in React.
- **Design Direction:** Dark navy/charcoal backgrounds with gold candlelight glows and indigo/purple accents; gothic-serif headings and a clean sans body font; minimal “magical” icons/particles. Animations should be subtle (fade-ins, hovers) and enhance rather than distract [18†L106-L114] [23†L16-L24] .

This MVP plan assumes a **5-hour hackathon**: finish core functionality first, then polish visuals.

Tech Stack

- **React + Vite:** Fast setup and hot-reloading. Vite is ideal for small teams and quick builds. [29†L61-L68]
- **React Router:** Single-Page App navigation between pages (Home, Events, etc.).
- **CSS Framework:** Tailwind CSS (or plain CSS modules) for rapid styling. Alternatively, a lightweight UI library (e.g. [Chakra UI](#) or [Mantine](#)) could work, but plain Tailwind ensures minimal bundle size.
- **Icons:** A clean icon set like [Lucide Icons](#) or [Feather Icons](#) for minimalist shapes (wand, star, scroll, etc.). Avoid overtly Potter-specific imagery.
- **Animations:** CSS transitions or [Framer Motion \(42 KB\)](#) for simple fades/slides. For particle effects, a tiny library like [tsParticles](#) (vanilla JS) can add a subtle starfield or floating embers without heavy 3D.
- **Fonts:** Google Fonts pairs. E.g.:
- **Display (serif):** *Cormorant Garamond*, *Cinzel*, or *Vollkorn* (for that gothic feel).
- **Body (sans):** *Inter*, *Manrope*, or *Montserrat* for readability.
(See [Google Fonts](#) for license-free usage.)
- **Hosting:** Netlify (free tier is fine). Use a `_redirects` or `netlify.toml` to redirect all routes to `index.html` (e.g. `/* /index.html 200`) so SPA routing works [29†L61-L68] [29†L99-L105] .

Do Not Include: No backend (no APIs), no database, no auth or payments, no server-side logic. We use *static/mock data* for everything [27†L4-L12] [27†L15-L22] .

Architecture & File Structure

Keep a clean React folder structure. Example:

```
nirvan26/                                (root project folder)
├── public/
│   ├── _redirects                        # for Netlify SPA routing
│   ├── index.html
│   └── images/                          # background, logos
├── src/
│   ├── assets/                          # images, icons, static media
│   │   ├── backgrounds/                 # e.g. castle.png, parchment textures
│   │   ├── gallery/                    # fest photos
│   │   ├── speakers/                   # portraits
│   │   └── sponsors/                    # logos
│   ├── components/                     # reusable UI components
│   │   ├── Navbar.jsx
│   │   ├── Footer.jsx
│   │   ├── Hero.jsx
│   │   ├── EventCard.jsx
│   │   ├── EventGrid.jsx
│   │   ├── EventFilter.jsx
│   │   ├── SpeakerCard.jsx
│   │   ├── SponsorGrid.jsx
│   │   ├── GalleryGrid.jsx
│   │   ├── ScheduleTimeline.jsx
│   │   └── RegisterModal.jsx
│   ├── pages/                          # route components
│   │   ├── Home.jsx
│   │   ├── About.jsx
│   │   ├── Events.jsx
│   │   ├── EventDetails.jsx
│   │   ├── Schedule.jsx
│   │   ├── Speakers.jsx
│   │   ├── Gallery.jsx
│   │   └── Contact.jsx
│   └── data/                           # static data files
│       ├── events.js
│       ├── speakers.js
│       ├── schedule.js
│       └── sponsors.js
```

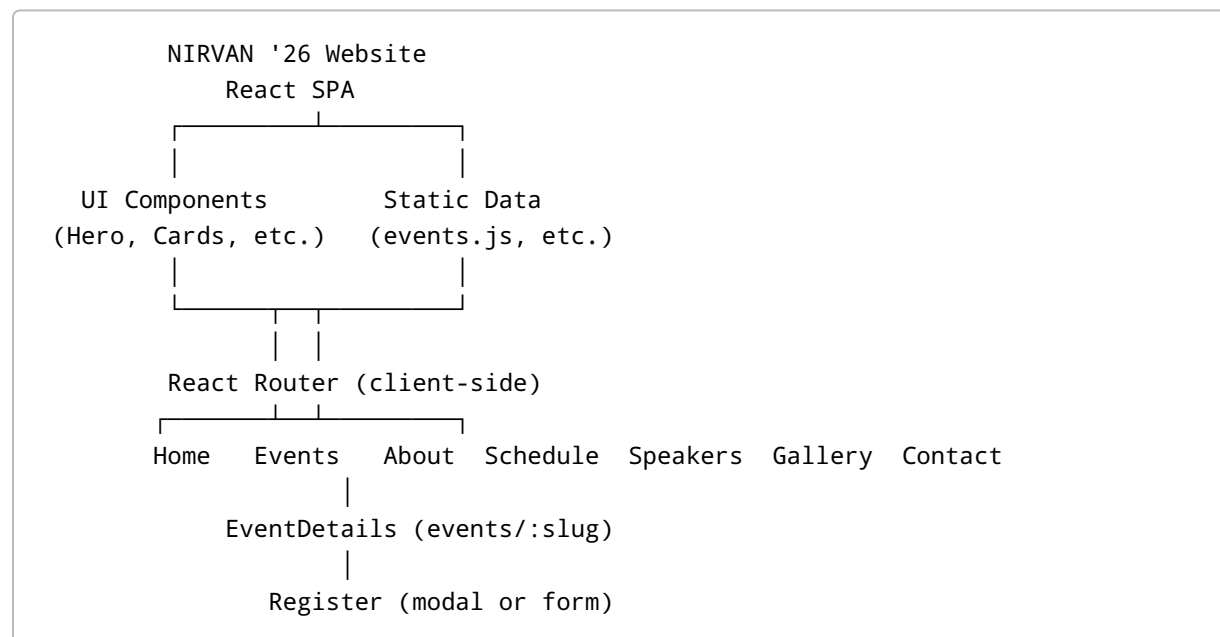
```

├── siteInfo.js    # fest name, tagline, contact info
├── styles/        # global CSS or Tailwind config
│   └── globals.css
├── App.jsx
├── main.jsx
├── package.json
└── netlify.toml

```

This separates *components*, *pages*, and *data*. For example, `events.js` might export an array of event objects (id, title, description, date, etc.), and `EventGrid.jsx` maps over them to render `EventCard` components. Keep content out of JSX! (Just as we planned: “content is separated from presentation.”) **【15†L9-L16】 【27†L15-L22】** .

A simple component diagram:



Page Contents (MVP Features)

We focus on **clarity and discoverability**. Based on the hackathon brief, each page should include:

- **Home (/)**: Hero (fest title, dates, tagline, two CTAs: “Explore Events” and “Register Now”), a brief fest description (what/why), highlights (past winners or stats), and previews of featured events or schedule. The hero can have a dark castle or library background with subtle floating particles.

- **About** (/about): Explain *what NIRVAN is* and *why it's organized*. Bullet/section this (mission, innovation, community). Use parchment-style info cards or columns. Cite any fest history or stats if available.
- **Events** (/events): List all competitions. Provide category filters/buttons (e.g. All, Hackathon, Workshop, CTF, E-sports, Treasure Hunt) 【27†L15-L22】 【20†L19-L27】 . Display event cards (title, short desc, date, entry fee) in a grid. Each card has a “View Details” button. Ensure the cards are scannable (consistent layout, clear labels).
- **Event Details** (/events/:slug): For each event: name, banner image, full description, date/time, venue, team size, eligibility, fee, prizes, rules list, and a prominent “Register Now” button 【15†L9-L16】 【15†L53-L61】 . Layout example: a two-column or stacked layout with key info boxes (Date, Time, Venue, Team Size, Fee, Prize) and text sections (Description, Rules). Add a “Back to Events” link. Keep it clean – unlike home, focus on info, not heavy graphics.
- **Schedule** (/schedule): Show a timeline or table of events by day and time 【20†L19-L27】 . Desktop: horizontal timeline or table; mobile: stacked cards per event. Example: “08:00 – Opening Ceremony, 10:00 – Hackathon, 14:00 – CTF, etc.” (from brief suggestions). You can implement a simple vertical timeline or list with times.
- **Speakers** (/speakers): Showcase at least 3 guest lecturers or judges, each with photo, name, title/role, and short bio 【15†L88-L96】 【15†L100-L108】 . Use consistent “portrait card” styling (e.g. circular frames, parchment nameplate).
- **Sponsors (could be section on Home or its own page)**: If included, show sponsor logos grouped by tier (Title, Gold, Community) 【18†L216-L224】 . Keep it subdued (e.g. gray background or footer area) so logos stand out without clashing with theme.
- **Gallery** (/gallery): A grid of ≥ 6 past-event photos 【18†L222-L232】 . Use a masonry or responsive grid. On hover or click, open a lightbox for larger view (a simple CSS modal or a small JS lib). Label images with captions/dates.
- **Contact** (/contact): List fest contacts: organizing team names (optional), general email, phone, and venue address 【27†L168-L176】 . If possible, embed a Google Map iframe for the campus location. Use icons (email, phone, map-pin).
- **Registration**: No backend form needed. Either (a) a dedicated page with a simple form that collects name/email and posts to a dummy endpoint (since static), or (b) a modal that links to an external form (if you have one) 【27†L4-L12】 . Most hackathon sites simply link “Register” to an external Google Form or event page. The key is having a clear **Register** button on Home, Events, and EventDetails.

All page text must be **readable and high-contrast**. Buttons should have meaningful labels (“Register Now”, “View Details”, etc.) 【15†L9-L16】 【27†L15-L22】 . Avoid vague CTA text.

Development Workflow (5-Hour Plan)

Time-box tasks to ensure a working MVP by the end:

1. Setup & Core Components (0:00–0:30):

2. Initialize project: `npm create vite@latest nirvan26 --template react`.

3. Install React Router and Tailwind (or CSS).
4. Set up `public/_redirects` or `netlify.toml` for routing **[29†L61-L68]** .
5. Create basic file structure (folders above).
6. Develop global `<Navbar>` and `<Footer>` with links to main pages and register CTA. Use a mobile-friendly hamburger menu.
7. Include theme fonts (e.g. via Google Fonts in `index.html`).

8. Home & About Pages (0:30–1:30):

9. Code the **Hero** component: background image overlay, title, tagline, date/venue, two buttons. (Test responsiveness.)
10. Below Hero, add an “About Nirvan” teaser and link to About page.
11. Add a **Featured Events** or “Why Participate” section (e.g. 3–4 columns with icons, referring to innovation, prizes, workshops).
12. Tease Schedule/Speakers/Gallery with short intros + links.
13. Fill **About** page with fest description (from brief or your own text). Break it into subsections.

14. Events List & Filter (1:30–2:30):

15. Create `events.js` data: an array of event objects (id, name, category, brief, date/time, etc.).
16. Build `<EventFilter>` (buttons or tabs) to filter by category (can be stateful in `Events.jsx`).
17. Build `<EventGrid>` to display `<EventCard>` s for each filtered event. Each card shows name, icon, short desc, date, and “View Details” link (e.g. React Router `<Link to={ /events/{slug} } / >`).
18. Implement `<EventDetails>` page: use `useParams()` to load the correct event data by `slug` . Render all required info and “Register Now” (which could scroll to a dummy form or open a modal).

19. Schedule, Speakers, Contact (2:30–3:15):

20. **Schedule:** Use a simple list or timeline component. Populate with sample data (from `schedule.js`). Make sure mobile view is legible.
21. **Speakers:** Define 3 sample speakers in `speakers.js` . Create `<SpeakerCard>` to display them in a grid.
22. **Contact:** Hard-code fest address/email/phone in `siteInfo.js` or `data` . Simple layout with labels and icons.

23. Gallery & Misc (3:15–3:45):

24. **Gallery:** Import a few images into `src/assets/gallery/` . Build `<GalleryGrid>` that maps them into responsive tiles. Add click-to-lightbox (you can use a tiny library like [React Image Lightbox](#) or a custom modal).
25. **Registration Modal/Form:** (If time) create a `<RegisterModal>` that pops up a form. Or at least ensure every “Register” CTA links to an actual action (e.g. `mailto:` or a placeholder).

26. Styling & Polish (3:45–4:30):

27. Apply the dark/gold color scheme consistently via CSS/Tailwind variables.

28. Add subtle animations: e.g. fade-in sections on scroll, hover glow on buttons/cards. (Framer Motion can handle this easily with `motion.div animate={{ opacity: 1 }}`.)

29. Add a particle background (e.g. [tsParticles](#) with a “floating embers” config) behind the Hero or footer for atmosphere, but keep it very light (e.g. ~50 small particles).

30. Optimize images (resize/compress) and lazy-load gallery images.

31. Responsive & Testing (4:30–5:00):

32. Test breakpoints: Desktop (1440px), Tablet (1024px), Mobile (390px) 【27†L15-L22】. Ensure nav collapses, text scales, images fit.

33. Check flows: clicking “Explore Events” scrolls or navigates to events, “View Details” → event page, “Register” appears on each event. Refresh on `/events/:slug` should not 404 (Netlify redirect).

34. Final run: `npm run build`, deploy to Netlify (connect repo or upload via Netlify UI). Confirm live site loads and nav works.

Remember: Judges prioritize a *working, clear site* over fanciness 【27†L4-L12】 【27†L15-L22】. Use time-saving tools (e.g. Tailwind classes) to reach a polished look quickly.

MVP Checklist (What’s in vs. out)

Included (must-haves):

- React SPA (Vite) with multiple pages via React Router.
- Static data files for events/speakers/schedule.
- Hero with CTA, clean navigation.
- Event listing with filtering and detail pages (all event info + register CTA).
- Schedule overview (timeline or list).
- Speakers page (≥3 profiles).
- Gallery (≥6 images) + lightbox.
- About page (fest description, purpose).
- Contact info (venue, email, phone).
- Mobile-responsive layouts (navigation menu, grid reflow, text legible).
- Basic animations (e.g. fade-ins, hover effects).
- Netlify-ready routing (`_redirects` or `netlify.toml`).

Excluded (to save time):

- **Backend/API/Auth:** Not needed (static only).
- **Payment system:** No registration fees to collect online.
- **Complex 3D/WebGL:** Only simple canvas effects if any.
- **PWA/Caching:** No time for ServiceWorker, etc.

- **Custom cursor / full parallax:** Too elaborate for 5 hours.
- **Multi-language:** English only.
- **Any heavy library:** Only minimal libraries (avoid 100KB+ animation libs).

Key Design/Data Decisions

- **Static JSON:** Example `events.js` entry:

```
export const events = [
  {
    id: 1,
    slug: "hackathon",
    title: "Hackathon",
    category: "Hackathon",
    description: "Teams innovate solutions to real-world problems.",
    date: "18 Dec 2026",
    time: "09:00 AM",
    venue: "Main Hall",
    teamSize: "2-4",
    eligibility: "All undergrads",
    fee: "₹100",
    prize: "₹15,000",
    rules: ["No plagiarism", "All work from scratch", "No external help"],
    registrationUrl: "https://forms.gle/...",
  },
  // ... other events
];
```

Then in `Events.jsx`:

```
import { events } from '../data/events';
return <EventGrid events={events} />;
```

and in `EventDetails.jsx`:

```
let { slug } = useParams();
let event = events.find(e => e.slug === slug);
```

- **Component Reusability:** Each UI part (card, section, header) is a React component. For example, an `<EventCard event={evt}/>` takes an event object and renders the card. This aligns with best practices of separation and reuse.
- **Responsive Menus:** Navbar collapses into a hamburger on small screens. EventFilter can become a

dropdown or accordion on mobile.

- **Accessibility:** Use semantic HTML (e.g. `<button>`, `<section>`) and alt text for images. Ensure color contrasts (e.g. gold text on dark navy).

- **Performance:** Compress images, import only needed icons. Animations triggered on scroll (via `IntersectionObserver` or a React hook) to avoid jank.

Deployment (Netlify)

1. **Build:** `npm run build`. Vite outputs `dist/`.
2. **Redirects:** Ensure either `public/_redirects` contains `/* /index.html 200`, or `netlify.toml` has:

```
[[redirects]]
from = "/*"
to = "/index.html"
status = 200
```

This makes Netlify serve the React app for all routes **[29†L61-L68]**.

3. **Publish:** Create a GitHub repo and connect it to Netlify, or drag `dist/` folder into Netlify. (Netlify will run `npm install && npm run build` if connected to repo.)

4. **Verify:** Visit the live URL. Test navigation and page refreshes (e.g. go to `/events`, refresh the page – it should load, not 404).

By following this plan, the **AI developer (or any dev on the team)** can quickly scaffold and build the NIRVAN '26 site. The focus is on **clear information architecture and a polished, lightweight UI**, using the Harry-Potter-inspired theme *only as a visual layer*. A working MVP with all required pages and flows will demonstrate understanding of the problem statement and satisfy the judging criteria **[27†L4-L12]** **[27†L15-L22]**.
